Tales of a transformation Journey at Proton



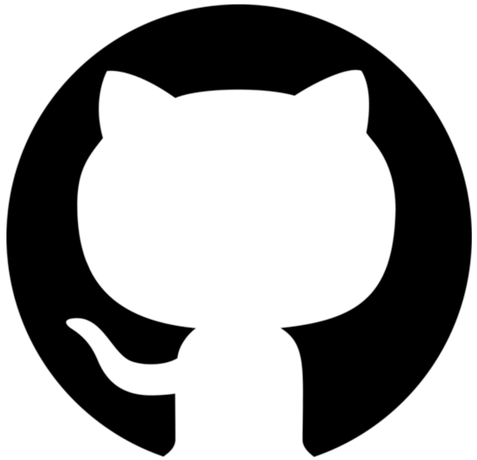


By Jeremy Lempereur

Jeremy Lempereur (o0ignition0o)



o0ignition0o.bsky.social



o0ignition0o

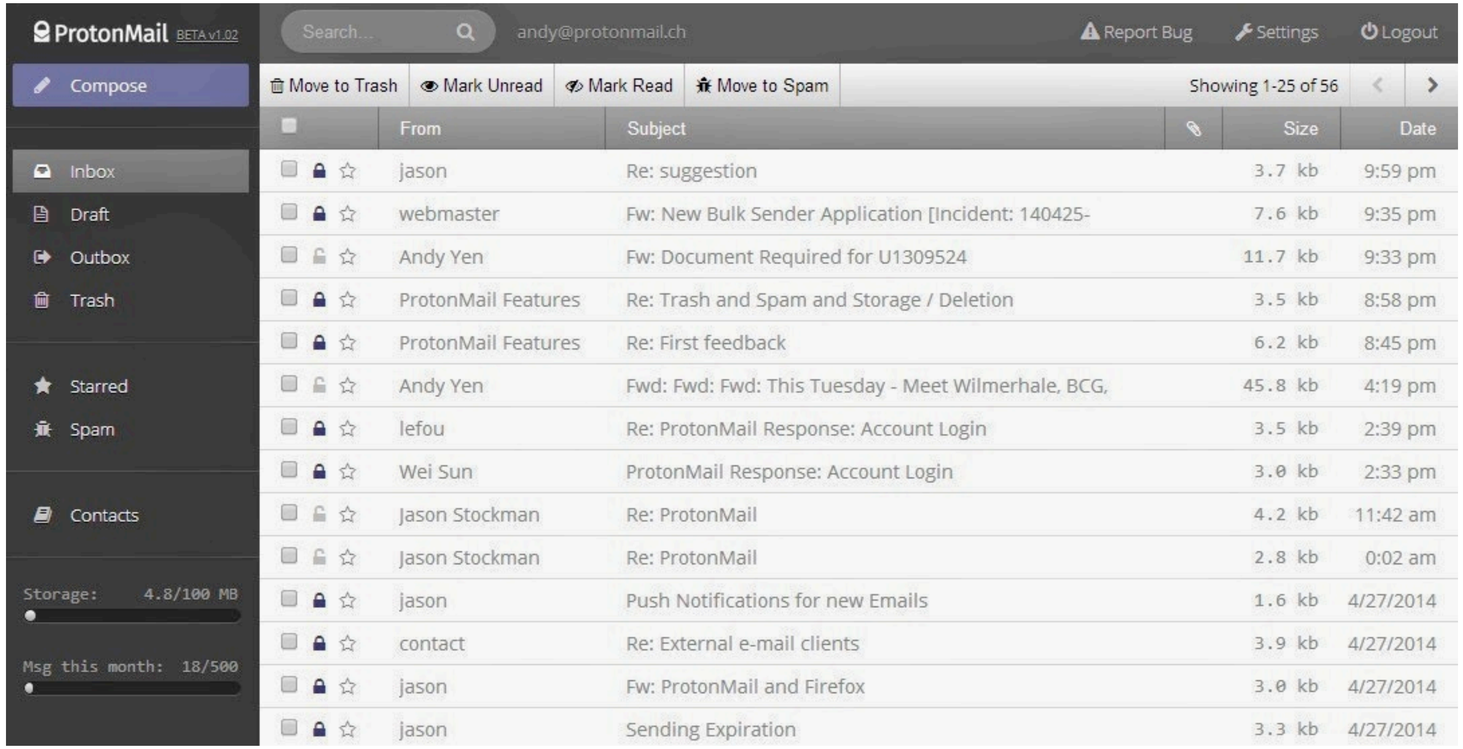


jeremy-lempereur

Jeremy

- Started using rust about 10 years ago
  - Thank you jdm for bugs ahoy!
- Did a bit of embedded
- Did a lot of web at scale
- Worked at ApolloGraphQL on federation and the Router
- Am now at Proton, working mostly on… clients!

Proton in 2014



ProtonMail for web

https://thehackernews.com/2014/05/protonmail-nsa-proof-end-to-end.html

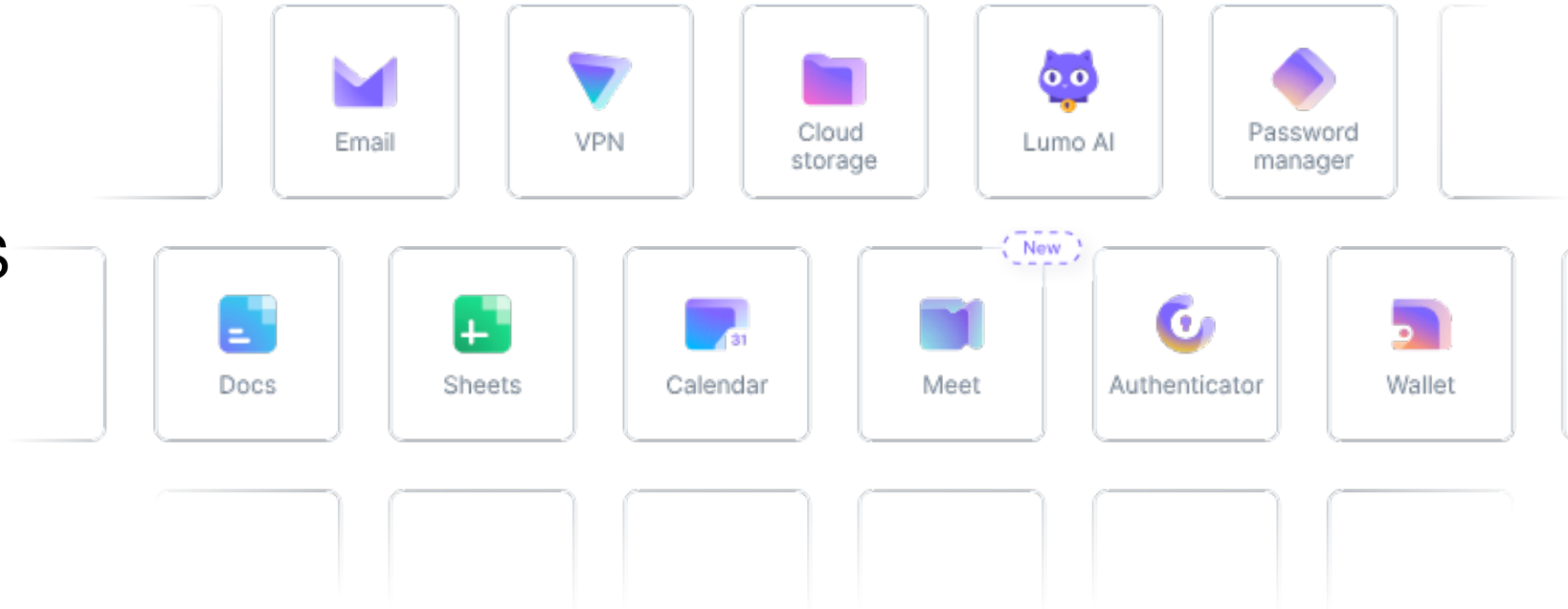
Proton in 2026

Web / extension

iOS / Android

Windows / Linux / MacOS

Android TV / Apple TV



… and more to come soon 🤫 🤫 🤫

Proton in 2026

More than 500 employees

50% engineering

We’re hiring btw!

https://proton.me/careers

How did we get there?

- Ruthless prioritization
- Focus on shipping
- Thanks to all of you!

Why did we need to change?

- Code duplication
- Difficult integration makes us miss deadlines
- Tech debt
- Organizational overhead

The objective

Enable

- Integration across components
- Integration across product features
- Portability (and yes, this includes web)

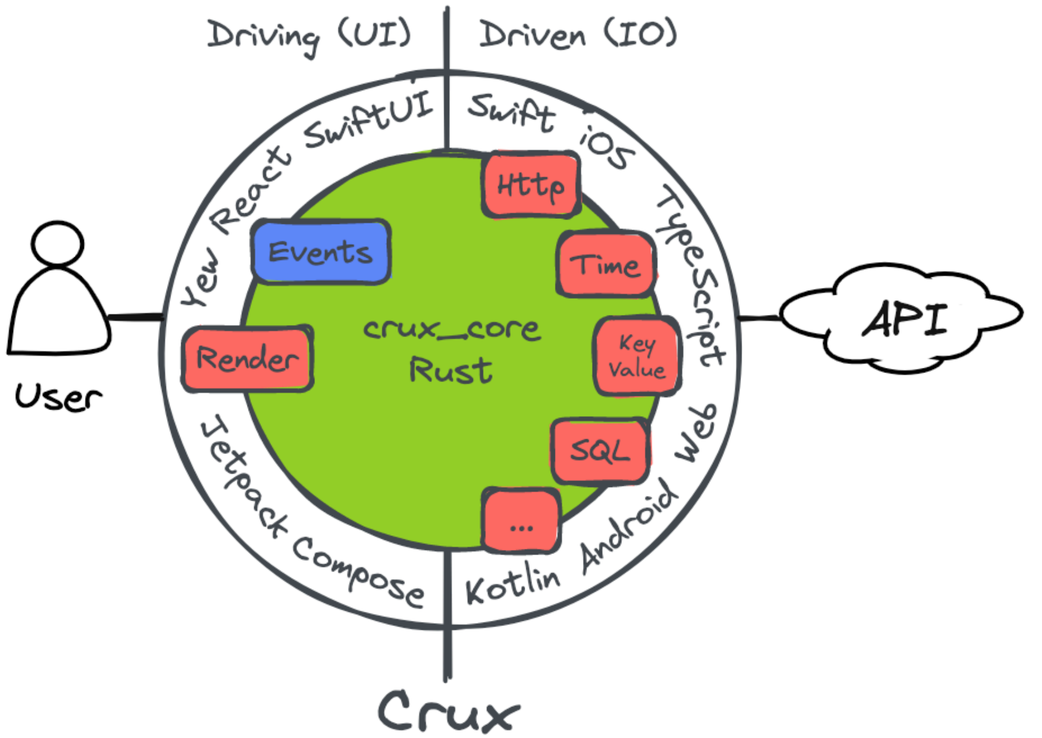
Avoid

- Removing agency from teams
- Extra integration overhead
- Wasting resources having several teams solve the same problem

A common architecture

Hexagon / Ports and adapters

- Push side effects to the edge (capabilities)
- Portable domain (core)
- Driven by a native layer (shell)



- The Public API Contract is all that matters

https://github.com/redbadger/crux/

Bindings

- UniFFI / wasm-bindgen
  - Update
  - Resolve
  - View

Codegen

- Facet
  - Events
  - Effects

```rust
#[derive(Facet, Serialize, Deserialize, Clone, Debug, PartialEq)]
#[repr(C)]
pub enum Event {
    Login(LoginInput),
    SendEmail(EmailToSend),
    Logout,
}

#[effect(facet_typegen)]
#[derive(Debug)]
pub enum Effect {
    Render(RenderOperation),
    Http(HttpRequest),
    Database(DatabaseOperation),
}

// boilerplate removed for brevity
fn update(&self, msg: Self::Event, model: &mut Self::Model) -> Command<Effect, Event> {
    match msg {
        Event::Login(credentials) => {
            // make http call
            // store result in db
            // render
        }
        Event::SendEmail(email_to_send) => {
            // make database call
            // http / smtp call
            // render
        }
        _ => {
            // etc.
        }
    }
}
```

Same pattern on the shell

```
Javascript shell side,
// Serialization removed for clarity

update(event: Event) {
  const effects = core.update(event);
  for (const { id, effect } of effects) {
    resolve(id, effect);
  }
}

async resolve(id: number, effect: Effect) {
  switch (effect.constructor) {
    case EffectVariantRender: {
      // update the view
      break;
    }
    case EffectVariantHttp: {
      // make an http call
      break;
    }
    case EffectVariantDatabase: {
        // perform a db operation
        break;
      }
    }
  }
```

```
Swift shell side,
// Serialization removed for clarity
func update(_ event: Event) {
    let effects = core.update(event)
    for request in effects {
        processEffect(request)
    }
}

func processEffect(_ request: Request) {
    switch request.effect {
    case .render:
        DispatchQueue.main.async {
            // update the view
        }
    case let .http(req):
        Task {
            // make an http call
        }
    case let .database(req):
        Task {
            // perform a db operation
        }
}
```

```
Kotlin shell side,
// Serialization removed for clarity
suspend fun update(event: Event) {
    val effects = core.update(event)

    for (request in effects) {
        processEffect(request)
    }
}

private suspend fun processEffect(request: Request) {
    when (val effect = request.effect) {
        is Effect.Render -> {
            // update the view
        }
        is Effect.Http -> {
            // make an http call
        }
        is Effect.Database -> {
            // perform a db operation
        }
    }
}
```

Common capability libraries

- Crypto
- API clients
- Debugging / Troubleshooting primitives

Common features

- Login / Signup
- Account management
- Feature flag support
- Cross product functionality

Common challenges

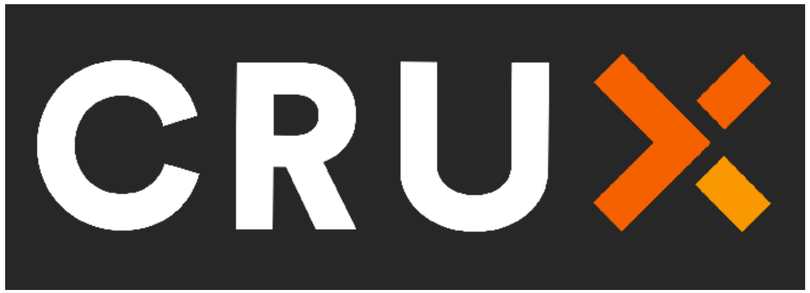
- Composition ✅
- Versioning and Packaging ⚠️
- Org wide buy in?

The road ahead

- Deeper hexagonal architecture adoption
- More cross product integration
- A unified cross product and cross platform experience

Thank you!







https://proton.me/careers

https://github.com/redbadger/crux

jeremy-lempereur